Recod.ai/LUC - Scientific Image Forgery Detection

Max Bakke Seymour[1], Einar Martin Søvik Bernsen[1], and Emre Balci[1]

[1]University of Bergen (UiB)

Abstract

This paper presents a model for the Recod.AI Scientific Image Forgery Detection Kaggle competition, and compares it with selected other architectures for Copy-Move Forgery Detection (CMFD). The comparison highlights different useful techniques for CMFD as well as dataset/task-specific considerations when performing CMFD. Although we were unable to test our method on the Kaggle test set due to late entry, we achieve a holdout oF1 of 0.5138 on a stratified hold-out test set derived from the Kaggle train set using a model combining DINOv2 features with BusterNet.

Additionally, we examine the role of strong pretrained feature extraction for this task by implementing a SegNeXt-inspired segmentation baseline and comparing it with the project's DINO-based baseline. The results show that using strong pretrained features is essential for CMFD.

1 Introduction

1.1 Copy-Move Forgery Detection

Copy-Move Forgery is a class of image forgery where regions of an image are duplicated and moved to other areas of the image. Commonly, techniques such as rotation, flipping and scaling of the copied region are applied, as well as post-processing such as edge smoothing, brightness adjustment and adding extra noise. These techniques can lead to forgeries which are hard to detect both through manual review and automated detectors. This method of forgery can be used within scientific images in order to fabricate results, where the complexity of the figures creates additional difficulties. [1]

1.2 Related works

Copy-move image forgery detection has been studied using a range of techniques. Early approaches relied on hand-crafted descriptors with nearest-neighbour matching and geometric consistency checks [2, 3], but are sensitive to strong post-processing, geometric transformations, and repeated structures [3].

More recent work focuses on deep learning. Some methods retain the matching paradigm with learned features [4], while others formulate the task as end-to-end forgery localization using CNN-based architectures [5]. Attention-based and transformer-based models further improve performance by modelling patch relationships and global context [6, 7]. Source-target disambiguation has also emerged as an important subproblem [8].

Some methods seek to combine the strengths of both approaches, such as one of the architectures examined in this paper which combines learned CNN features with Zernike-based descriptors [9].

1.3 Objectives

We aim to create a full prediction pipeline for CMFD which could perform well on the Recod.AI Scientific Image Forgery Detection dataset. Ideally, our model should be robust towards post-processing techniques. The main goal is to perform well on the competition metric, which is a modified version of F1 which computes pixelwise F1 on pairs of (*predicted mask, ground truth mask*) using the following formula [10]:

$$F1 = \frac{2TP}{2TP + FP + FN}$$

This calculation is applied to every connected component of forged pixels in the image (belonging to either the source or target). Predictions given by the model are matched up with the best corresponding ground-truth areas using the Hungarian algorithm. For any additional areas predicted by the model above the actual amount of ground-truth components, a penalty is computed:

$$P = \frac{n_{gt}}{\max(n_{pred}, n_{gt})}$$

Finally, the per-image score is given as:

$$oF1 = \text{mean(best matched pairwise F1 scores)} \times P$$

The competition metric is challenging for a few reasons. Firstly, it requires high pixel-level precision for the output masks. In addition, a score of 0 for a given image is given if any pixels are predicted as forged on a ground-truth authentic image. Finally, it is highly punishing if any additional components are predicted, even if they should be small, as this affects the penalty P.

1.4 Contributions

This paper presents several complete pipelines for CMFD on the Recod.AI Scientific Image Forgery Detection dataset. The methods are all fundamentally based on DINOv2 segmentation features [13], where the baseline passes the features directly through a decoder. To build on this, we also present more complex architectures based on BusterNet [5] and Cross-Scale PatchMatch [9]. The strengths and weaknesses of these approaches are discussed, providing an empirical basis for comparing different design choices. Through this analysis, the paper identifies directions that appear promising for future work and further nuances the task of CMFD.

2 Methods

We run two experiments. Firstly, our main experiment assesses the performance of the different methods based on DINOv2 features. All experiments were conducted with a fixed random seed of 42. We also seeded Python, NumPy, PyTorch, CUDA, and data-loader workers where applicable, and disabled cuDNN benchmarking to make the runs highly reproducible. However, we do not claim strict bit-wise determinism, as some PyTorch/CUDA operations and architecture-dependent kernels can remain nondeterministic in practice, and enforcing fully deterministic algorithms would have made parts of the training pipeline impractical.

Our secondary experiment evaluates feature extraction strength of DINOv2, by comparing it against a pretrained SegNeXt-inspired model for different dataset sizes, measuring validation F1, without applying any post-processing in order to keep this experiment clean.

2.1 Dataset

We utilize the dataset supplied in the Kaggle competition [1]. Unfortunately, we were unable to test our implementation against the Kaggle test-set, as we were too late to enter the competition. In our main experiment, we chose to use the available training data in a train (80%), evaluate (10%), test (10%) split. In our secondary experiment, we use different-sized subsets of the same dataset in order to assess the impact of dataset size against feature extraction. The training data consists of a base 2751 forged and 2377 authentic images as well as an additional 48 forged images. As the images are too large to process directly with the model, they are each resized to size $3 \times 448 \times 448$.

2.2 Training Objective and Optimization

All models are trained as a pixel-wise binary classification problem, where each pixel is classified as either forged or authentic. We use binary cross-entropy loss for pixel-wise supervision, and utilize the Adam optimizer with a learning rate $\alpha = 0.0001$:

$$\mathcal{L}_{BCE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

2.3 Dino-Segmentation Model

As the Kaggle competition had already been running for 3 months when we began our experiments, our initial implementation was inspired by the techniques used by other users [11, 12]. We noted that many users employed the DINOv2 model [13], as well as techniques for prediction mask post-processing. Our main model is inspired by the use of these techniques.

2.3.1 Feature Extraction

Within our main method we employ Meta AI's powerful DINOv2 model using the pretrained *ViT-B/14* segmentation head [13] in order to perform feature extraction on the images. This model first extracts strong general-purpose features using the DINOv2 ViT backbone, then uses these features to create pixel-level segmentation predictions using the segmentation head. This model is kept frozen and applied on 32×32 size patches of the image at a time to generate features with an embedding dimension of 768. When predicting, the model processes image patches independently and reconstructs the final prediction by aggregating patch-level outputs.

2.3.2 Decoder

To generate the predicted mask, we use a lightweight convolutional decoder on top of the DINOv2 feature map. The decoder consists of three convolutional blocks followed by an output layer, and produces a mask upsampled to the input resolution using bilinear interpolation.

The feature dimensionality is reduced progressively from $768 \to 384 \to 192 \to 96$, with ReLU activations and Dropout ($p = 0.1$) applied in each block, before a final $96 \to 1$ projection.

Let $X \in \mathbb{R}^{3\times H\times W}$ and $F \in \mathbb{R}^{768\times h\times w}$ denote the input and extracted features. The predicted mask is:

$$\hat{Y} = \sigma(\text{Decoder}(F))$$

2.4 Deep Cross-Scale PatchMatch

The Deep Cross-Scale PatchMatch architecture [9] combines two branches: a PatchMatch-based module [14] that estimates pixel correspondences across scales, and a direct feature extraction → prediction branch. Their outputs are combined via an argmax operation.

We adapt the architecture to single-channel CMFD and use only the first branch. To reduce computational cost, we replace the original feature extractor with a frozen pretrained CNN, avoiding backpropagation through the PatchMatch module. We evaluate both VGG16 and ResNet18 backbones [15, 16], with ResNet18 performing best. We further simplify the model by using single-scale CNN features while retaining multi-scale Zernike features. Inspired by our baseline, we use DINOv2 for feature extraction in the second branch.

In addition to the original dice losses for the final mask M and SEUnet output M_t, we include the binary cross-entropy used in the baseline loss as well as two auxiliary terms: (i) a dice loss on the PatchMatch output M' to prevent collapse, and (ii) a penalty on predictions for authentic images to reduce false positives. These are weighted by 0.8 and 0.25, respectively.

2.5 Modernized BusterNet implementation

The modern implementation of BusterNet [5] uses the core concepts of a similarity layer, a two-branch approach with a fusion model, and a 3 stage training setup. The inner architecture was adapted to work with the baseline's DINOv2 as a feature extractor and modern deep learning principles for efficiency and robustness. The paper made it explicit that the core concepts were most important and inner implementations could be changed by the reader. Therefore it was most fair to revise it to be comparable with the baseline implementation too using modern decisions. For more implementation details check the implementation [18]

2.6 SegNeXt-inspired implementation

SegNeXt is motivated by the idea that semantic segmentation benefits from strong encoders, multi-scale feature interaction, and efficient spatial attention implemented with convolutional operations rather than transformer self-attention [17]. It employs a convolutional encoder–decoder architecture with multi-scale convolutional attention to capture contextual information efficiently, achieving strong performance–computation trade-offs and outperforming transformer-based alternatives on several benchmarks [17]. We use SegNeXt to evaluate whether a lightweight CNN-based approach can match the stronger DINO-based baseline.

Our goal was not to reproduce the full SegNeXt architecture, but rather to implement a simplified, SegNeXt-inspired baseline within our pipeline. The model follows a standard encoder–decoder design, emphasizing convolutional feature extraction and multi-scale context aggregation. This lightweight setup makes it feasible under project constraints while remaining relevant for scientific forgery localization. Additionally, we evaluate a pretrained variant using a `convnext_tiny` backbone from `timm` to assess the impact of stronger pretrained visual features.

2.7 Mask post-processing

Post-processing steps were critical for performance under the competition metric. In particular, removing small connected components significantly improved the oF1 score by reducing false positives on authentic images. In the main method, we employ the following post-processing steps in order: gaussian blur, confidence threshold filtering based on the threshold 0.5, closing with a 5×5 structuring element, opening and binary filling. The gaussian blur, closing, opening and binary filling operation are all implemented through the `scipy.ndimage` library. The PatchMatch implementation runs similar post-processing but first applies filtering of small components in order to reduce false positives.

3 Results

3.1 Main results

At the time of writing, the maximum oF1 score achieved on hold-out test data was 0.458, achieved by Kaggle user *orshanec*. [1] We again emphasize that our architecture cannot be directly compared with this metric due to us being unable to access the Kaggle hold-out test set.

3.1.1 Main model results

Our main model achieved an oF1 score of 0.5007, with F1 of 0.9958 on authentic and 0.0722 for forged. We observed that it was reluctant to make predictions. This indicates that during training the loss of predicting wrong on authentic images hurts so much that we instead rarely make a prediction. The performance measure discussed earlier supports this approach. We are using a stratified holdout set with 238 authentic and 275 forged. The semi-balance in classes helps, but if the holdout were to be dominated by forged samples it would perform worse.

We think given the performance measure, they are instead encouraging low FP rate for authentic samples, which the baseline achieves. It will instead fire when it is certain. Furthermore we believe that the simplicity of the decoder is holding back the results for the forged predictions. It is not utilizing the extracted features much, instead downsampling to a binary mask. If it were to upsample and do more convolutions it could perhaps learn more, especially on smaller forgeries. Right now it is mainly looking for large certain forgeries that are low risk. The runtime of the baseline is quite fast. The DINOv2 extractor dominates runtime as the decoder is smaller by a significant margin. It was trained for around 15 minutes on a 4080 super.

3.1.2 Deep Cross-Scale PatchMatch results

The PatchMatch architecture achieved competitive performance, reaching 0.613 on authentic images and 0.340 on forged images, giving a combined score of 0.466. However, this architecture had much higher runtime and memory consumption compared to our main method. The runtime is particularly dependent on the number of PatchMatch iterations which are performed. This module iteratively executes propagation and evaluation steps "several times" [9, p. 7]. While the original paper does not specify the number of iterations, we found that 24 iterations produced stable results. Using this amount of iterations, PatchMatch on a single image takes 4s on an NVIDIA RTX 4070 TI Super GPU, while other components of the network take less than $0.1s$ to run.

Achieving good performance with this approach was highly dependent on mask post-processing.. In particular, it is crucial to perform filtering of small objects, as the architecture generated many small false positive areas on authentic images, automatically giving an oF1 score of 0 in these cases. We theorize that this tendency to predict false positives is because smaller groups of pixels are more likely to be similar to each other "by accident" than larger groups of pixels. The best version of the pipeline on validation data filters out components with a size less than 512. Although this means accepting that the model will never be able to deal with very small copy-move forgeries, it led to strong performance on average due to the removal of many false positives.

3.1.3 BusterNet results

BusterNet-DINO achieved the strongest holdout oF1 among the three architectures. Results are summarised in Table 1.

	DINO baseline	SegNeXt-inspired	SegNeXt-family
Encoder	ViT-B/14	custom	convnext_tiny
Pretr.	Yes	No	Yes
Train	900	500	900
LR	3e-5	3e-5	3e-5
Ep.	8	5	12
Best Ep.	8	1	10
Val F1	**0.1850**	0.0838	0.1827

Table 2. Best configurations for the compared model families

Metric	Value
Holdout oF1	0.5138
Authentic mean F1	0.8361
Forged mean F1	0.3292
Forged empty rate	31.6%
Forged precision (non-empty)*	0.719
Forged F1 (non-empty)*	0.495

Table 1. BusterNet-DINO results. *Non-empty metrics computed on validation forged images with a non-empty prediction.

The model remained conservative with a forged empty rate of 31.6%, but produced high-quality masks when predictions were made. We attribute this to the two-branch structure. When the manipulation-detection and copy-detection branches agree, the fusion head becomes confident. When they diverge, it tends toward an empty prediction. The target masks are typically stronger than source masks, preserving conservatism while still producing accurate forged mask localisation. The added branch complexity also made BusterNet-DINO less dependent on aggressive post-processing, as the model better distinguishes noise from genuine small forgeries. See Figures A.1–A.3 in the appendix for probability distributions by size bucket and example predictions.

BusterNet-DINO is likely undertrained given the hard epoch cutoff. A characteristic pattern emerged during stage-wise training that validation oF1 initially decreased in stages 2 and 3 as the model began predicting more forged pixels, reducing authentic F1. However, inspection of the predictions showed that forged mask quality improved substantially in later checkpoints, yielding a stronger balanced model overall. The most balanced version was the one chosen for the results, trained across 40 epochs (20/10/10 per stage, ≈1h on an RTX 4080 Super).

3.2 Comparison Between DINO and SegNeXt-inspired Models

Overall, the DINO-based baseline consistently achieved the highest validation performance across

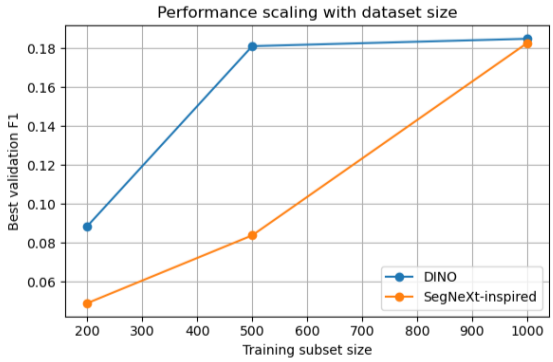

Figure 1. Validation F1 versus dataset size. The DINO baseline performs better at small dataset sizes, while the pretrained SegNeXt-family variant approaches it at the largest setting.

all tested configurations. The pretrained SegNeXt-family model approached this performance, but required more training epochs and relied heavily on pretrained representations. Learning rate had a clear effect on final validation performance, giving much stronger results for both methods for 3×10^{-5} than for 10^{-5} for both models.

We also attempted using a custom SegNeXt-inspired model without pretraining, but this performed substantially worse, with best validation F1 values remaining below 0.09 across all tested settings. This custom model often reached its best validation F1 already at the first epoch, which suggests unstable optimization and weak feature learning without pretraining, indicating the importance of pretrained visual representations.

4 Discussion

Transfer learning. Transfer learning played a central role in our experiments. The pretrained SegNeXt-family model approached the DINO baseline, while models trained from scratch performed substantially worse. This suggests that, under limited data, representation quality is a primary bottleneck, and strong pretrained encoders are crucial for effective CMFD.

Loss function misalignment. Although cross-entropy loss correctly penalizes incorrect pixel-level predictions, it is not fully aligned with the strict conditions that the oF1 metric expects. For instance, if a small area of forged pixels is predicted on an authentic image, the impact on the loss is not large, but this makes the oF1 metric 0. We observed that validation loss could decrease while oF1 performance worsened, indicating a mismatch between the training objective and evaluation metric. A promising direction for future work is to design a differentiable loss more closely aligned with the oF1 objective. As surrogates, we did experiment with employing loss penalties for any forged pixels predicted on authentic images, but a more nuanced alternative would be interesting to experiment with. For BusterNet adding the DICE function in a weighted sum with BCE improved performance greatly compared to regular BCE. This is an example of a problem specific loss function that accounts both pixel overlap and binary label.

Post-processing. Because of the loss function mismatch, model performance is highly sensitive to post-processing, making it an important direction for further optimization in future work. For instance, filtering out small predicted patches led to much better results, as the oF1 metric also penalizes extra component predictions on forged images. Other pitfalls are combining pixels too aggressively, causing large connected patches where the true prediction should consist of multiple smaller patches.

Failure modes. A major failure mode observed across all models was false positive predictions on authentic images. Due to the strict evaluation metric, even a small predicted region results in an oF1 score of zero. This makes the task highly sensitive to small prediction artifacts.

Dataset. We attribute part of the difficulty to dataset characteristics. Authentic images often contain visually similar structures, increasing the likelihood of false positives. While many forgeries involve large, consistent objects, some occur in small background regions, making detection more challenging.

For instance, PatchMatch's reliance on finding groups of pixels with high feature similarity may have some other fundamental issues in regards to this particular dataset. In the paper, the architecture utilized by Li et al. trains on a training set generated from MS COCO [19] by applying a random copy-move, rotation and scaling operation to a random area. It is then evaluated on a synthetic dataset generated in the same way as well as CASIA CMFD, CoMoFoD and CMH [20–22]. The common denominator for these datasets is that they feature photo-realistic images with varied objects and backgrounds. This means that there is natural variation in the qualities of pixels (for instance, they do not feature a consistent single-colour background) in contrast to many of the scientific images, which are dark, or figures containing a consistent background in which many false positives are generated from pixel-wise similarities. On the images where the copy-move is direct and

not subject to post-processing, our architecture performed much better.

References

[1] Kaggle. *Recod.ai/LUC - Scientific Image Forgery Detection*. https://www.kaggle.com/competitions/recodai-luc-scientific-image-forgery-detection/overview. [Online; last accessed April-2026]. 2025.

[2] J. Fridrich, D. Soukal, and J. Lukáš. "Detection of Copy-Move Forgery in Digital Images". In: (2003). URL: https://ws2.binghamton.edu/fridrich/Research/copymove.pdf.

[3] V. Christlein, C. Riess, J. Jordan, C. Riess, and E. Angelopoulou. "An Evaluation of Popular Copy-Move Forgery Detection Approaches". In: *IEEE Transactions on Information Forensics and Security* 7.6 (2012), pp. 1841–1854. DOI: 10.1109/TIFS.2012.2218597. URL: https://arxiv.org/abs/1208.3665.

[4] Y. Liu, C. Xia, X. Zhu, and S. Xu. "Two-Stage Copy-Move Forgery Detection With Self Deep Matching and Proposal SuperGlue". In: *IEEE Transactions on Image Processing* 31 (2022), pp. 541–555. DOI: 10.1109/TIP.2021.3132828. URL: https://doi.org/10.1109/TIP.2021.3132828.

[5] Y. Wu, W. Abd-Almageed, and P. Natarajan. "BusterNet: Detecting Copy-Move Image Forgery with Source/Target Localization". In: *Computer Vision – ECCV 2018* (2018), pp. 170–186. URL: https://openaccess.thecvf.com/content_ECCV_2018/papers/Rex_Yue_Wu_BusterNet_Detecting_Copy-Move_ECCV_2018_paper.pdf.

[6] A. Islam, C. Long, A. Basharat, and A. Hoogs. "DOA-GAN: Dual-Order Attentive Generative Adversarial Network for Image Copy-Move Forgery Detection and Localization". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2020), pp. 4676–4685. DOI: 10.1109/CVPR42600.2020.00473. URL: https://doi.org/10.1109/CVPR42600.2020.00473.

[7] Y. Liu, C. Xia, S. Xiao, Q. Guan, W. Dong, Y. Zhang, and N. Yu. "CMFDFormer: Transformer-based Copy-Move Forgery Detection with Continual Learning". In: *CoRR* abs/2311.13263 (2023). DOI: 10.48550/ARXIV.2311.13263. URL: https://arxiv.org/abs/2311.13263.

[8] M. Barni, Q.-T. Phan, and B. Tondi. "Copy Move Source-Target Disambiguation Through Multi-Branch CNNs". In: *IEEE Transactions on Information Forensics and Security* 16 (2021), pp. 1825–1840. DOI: 10.1109/TIFS.2020.3045903. URL: https://doi.org/10.1109/TIFS.2020.3045903.

[9] Y. Li, Y. He, C. Chen, L. Dong, B. Li, and J. Zhou. "Image Copy-Move Forgery Detection via Deep PatchMatch and Pairwise Ranking Learning". In: (2024). Also available at: https://arxiv.org/pdf/2404.17310. URL: https://ieeexplore.ieee.org/document/10736426.

[10] Kaggle. *RecodAI F1*. https://www.kaggle.com/code/metric/recodai-f1/. [Online; accessed January-2026]. 2025.

[11] G. Parkhedkar. *DINOv2 Base [0.332] — High-Res 4500px — Robust Inf.* https://www.kaggle.com/code/gauravparkhedkar/dinov2-base-0-332-high-res-4500px-robust-inf. [Online; accessed January-2026]. 2026.

[12] M. Ravaghi. *Scientific Image Forgery Detection — DINOv2*. https://www.kaggle.com/code/ravaghi/scientific-image-forgery-detection-dinov2. [Online; accessed January-2026]. 2026.

[13] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. "DINOv2: Learning Robust Visual Features without Supervision". In: *arXiv preprint arXiv:2304.07193* (2023). implementation: https://github.com/facebookresearch/dinov2. URL: https://arxiv.org/abs/2304.07193.

[14] C. Barnes, E. Shechtman, A. Finkelstein, and D. B. Goldman. "PatchMatch: A Randomized Correspondence Algorithm for Structural Image Editing". In: (2009). URL: https://dl.acm.org/doi/10.1145/1531326.1531330.

[15] K. Simonyan and A. Zisserman. "Very Deep Convolutional Networks for Large-Scale Image Recognition". In: *arXiv preprint arXiv:1409.1556* (2014). URL: https://arxiv.org/abs/1409.1556.

[16] K. He, X. Zhang, S. Ren, and J. Sun. "Deep Residual Learning for Image Recognition". In: *arXiv preprint arXiv:1512.03385* (2016). URL: https://arxiv.org/abs/1512.03385.

[17] M.-H. Guo, C.-Z. Lu, Q. Hou, Z.-N. Liu, M.-M. Cheng, and S.-M. Hu. "SegNeXt: Rethinking Convolutional Attention Design for Semantic Segmentation". In: *arXiv preprint arXiv:2209.08575* (2022).

[18] E. M. S. Bernsen. *BusterNet-DINO Implementation*. https://github.com/ei1000/INF367A-26V-Recod.ai-LUC-Forgery-Kaggle/tree/main/einar_busternet. [Online; accessed April-2026]. 2026.

[19] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. "Microsoft COCO: Common Objects in Context". In: *European Conference on Computer Vision (ECCV)*. 2014, pp. 740–755.

[20] J. Dong, W. Wang, and T. Tan. "CASIA Image Tampering Detection Evaluation Database". In: *IEEE China Summit and International Conference on Signal and Information Processing*. 2013, pp. 422–426.

[21] D. Tralic, I. Zupancic, S. Grgic, and M. Grgic. "CoMoFoD — New Database for Copy-Move Forgery Detection". In: *Proceedings of the International Symposium ELMAR*. 2013, pp. 49–54.

[22] E. Silva, T. Carvalho, A. Ferreira, and A. Rocha. "Going Deeper into Copy-Move Forgery Detection: Exploring Image Telltales via Multi-Scale Analysis and Voting Processes". In: *Journal of Visual Communication and Image Representation* 29 (2015), pp. 16–32.

A BusterNet-DINO Diagnostics

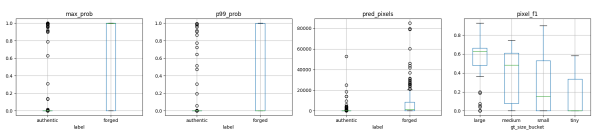

Figure A.1. BusterNet-DINO validation distributions. Left to right: maximum predicted probability per image (authentic vs. forged), 99th-percentile probability, total predicted pixels, and pixel F1 by ground-truth mask size bucket. Forged images show bimodal behaviour — the model either fires confidently or produces an empty prediction. F1 degrades for smaller forgeries, consistent with the size-bucket summary in the main text.

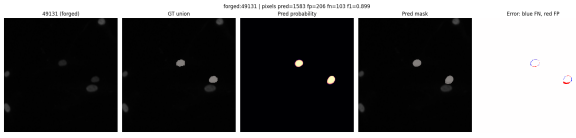

Figure A.2. BusterNet-DINO successful prediction on a microscopy image (case 49131, F1=0.899). From left: forged image, ground-truth union mask, predicted probability map, predicted binary mask, and error overlay (blue = false negative, red = false positive). The model correctly localises both the source and target regions with minimal error.

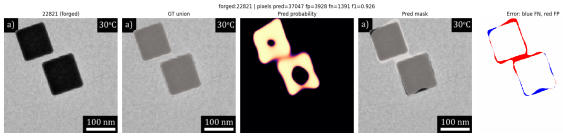

Figure A.3. BusterNet-DINO prediction on a nanoparticle image (case 22821, F1=0.526). The model identifies the copied rectangular region; residual errors at the edges are visible in the error overlay. This example illustrates the challenge of precise boundary localisation on high-contrast scientific images.

B PatchMatch Post-Processing Performance Gap

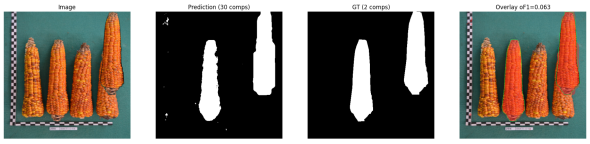

Figure B.1. PatchMatch on an image using no post-processing, achieving oF1=0.063. The model detects the copied object but also generates many small components. This affects the oF1 penalty P, leading to an overall low oF1 score, even though per-pixel precision is quite strong.

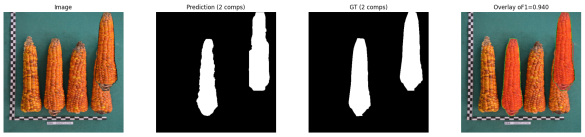

Figure B.2. PatchMatch on the same figure using post-processing, achieving oF1=0.948

C Dataset examples

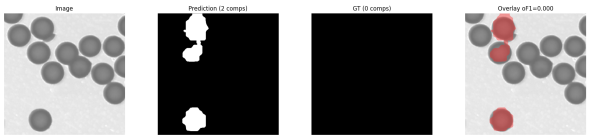

Figure C.1. An authentic image containing very similar objects, causing false positives in the Cross-Scale PatchMatch prediction.

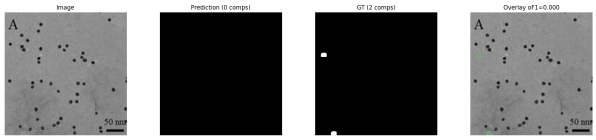

Figure C.2. A forged image with the source being hard to predict due to size. The aggressive minimum component size post-processing performed by some of the models examined in this paper, such as Cross-Scale PatchMatch, mean that they would be unable to detect this forgery.

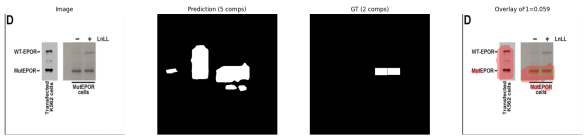

Figure C.3. A scientific image where mask post-processing combines components too aggressively. In addition, this highlights some challenges due to dataset characteristics - the text ("MutEPOR") is predicted as a forgery because it appears in the exact same form twice.